

Automatically Proving Commutativity to Improve Transaction Chopping

Yanjun Chen
University of California, Irvine
yanjunc6@uci.edu

Juncheng Fang
University of California, Irvine
junchf1@uci.edu

Farzad Habibi
University of California, Irvine
habibif@uci.edu

Brian Demsky
University of California, Irvine
bdemsky@uci.edu

Faisal Nawab
University of California, Irvine
nawabf@uci.edu

ABSTRACT

Distributed transactions often aim to ensure *serializability*, which degrades performance. Transaction chopping protocols improve concurrency by breaking transactions into smaller “hops”. However, chopping protocols’ effectiveness depends on how they build a dependency graph between transactions, called the *SC-graph*. Manually identifying commutative operations to prune this graph is complex and error-prone.

In this paper, we introduce *VIC3*, a system that automates the construction of a precise, correct SC-graph by integrating formal verification. Our framework formally models transaction semantics and leverages an SMT solver to automatically prove whether conflicting hops are commutative, allowing for the safe removal of extraneous dependency edges. This process systematically breaks cycles that arise from conservative analysis, potentially enabling more concurrency. Our evaluation shows that our system correctly identifies non-trivial commutative operations. The experiment result shows 1.6× improvement on the scaled TPC-C benchmark over the IC3 protocol without automated commutativity analysis.

PVLDB Reference Format:

Yanjun Chen, Juncheng Fang, Farzad Habibi, Brian Demsky, and Faisal Nawab. Automatically Proving Commutativity to Improve Transaction Chopping. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/yanjunc6/FMitF_rs.

1 INTRODUCTION

To guarantee correctness in distributed data-intensive applications, database systems must ensure that transactions are serializable. A common method to achieve serializability is strict two-phase locking (2PL), which holds locks across all nodes for the entire transaction duration. However, this approach severely limits concurrency by serializing transactions that could otherwise run in parallel, thereby degrading system performance.

A key strategy to address this bottleneck is to break, or chop, a transaction into smaller execution units, termed *hops* [31]. As illustrated in Figure 1, conventional transaction-level locking prevents any interleaving between transactions if a conflict exists (Figure 1a). In contrast, hop-level locking acquires and releases locks for each hop, allowing hops from different transactions to interleave (Figure 1b). This fine-grained locking can significantly improve concurrency and throughput. However, ensuring serializability with such interleaving requires a detailed dependency analysis.

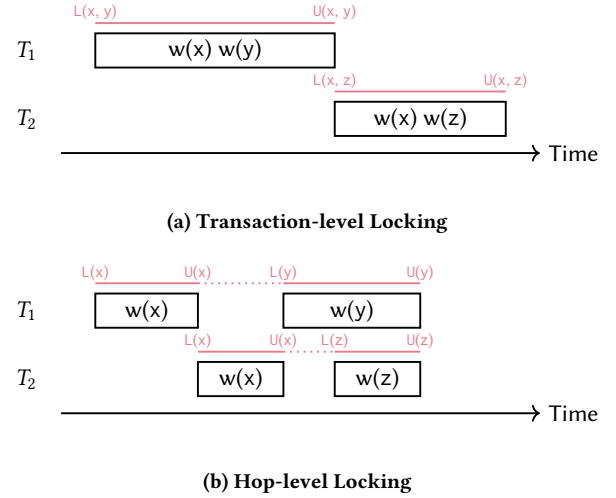


Figure 1: A conceptual depiction of locking strategies. The red line represents lock hold time. $L(x)$ and $U(x)$ represent lock and unlock operations on data item x . In (a), locks are held for the entire transaction. In (b), locks are acquired and released per hop, allowing hops from different transactions to interleave.

The foundation for this analysis is the SC-graph, a graph whose nodes are transaction hops and whose edges represent dependencies. The SC-graph contains two edge types: S-edges connect successive hops within the same transaction, while C-edges connect potentially conflicting hops from different transactions. A cycle in this graph involving both S-edges and C-edges, or an SC-cycle, indicates a potential serializability violation. Although early theory requires the chopping to have no SC-cycles, modern protocols

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

such as IC3 [35] guarantee serializability even in the presence of SC-cycles. Nonetheless, the performance of these protocols still depends on the quality of the initial SC-graph; an imprecise graph with unnecessary C-edges might lead to worse performance.

We use the running example in Figure 2 to illustrate. The system has two transactions: T_1 and T_2 , which operate on a stock table S . The transactions are expressed in TransAct, a domain-specific language we introduce for expressing chopped transactions. While our implementation uses TransAct, our modeling and analysis techniques are general. An automated tool can construct an SC-graph via conservative syntactic analysis, for instance, by adding a C-edge between any two hops from different transactions that access the same table where at least one access is a write. Since all hops in Figure 2a modify table S , the resulting SC-graph is densely connected with C-edges, as shown in Figure 2b. This graph contains numerous SC-cycles, which would cause a protocol like IC3 to frequently stall or merge transactions.

The key insight is that not all of these C-edges are semantically necessary. If two conflicting hops commute—that is, their execution order does not affect the final state or their return values—the C-edge between them is redundant and can be safely removed. Pruning such edges can reduce SC-cycles, as shown in Figure 2c, thereby enabling more concurrency. However, identifying commutativity is challenging. The commutativity between hop $T_{1,1}$ (a decrement) and $T_{2,2}$ (an increment) is trivial. In contrast, proving that two instances of hop $T_{2,1}$ commute is non-trivial due to the operation’s conditional logic. Although empirical verification with concrete values might show that the final database state remains consistent, this approach is not a formal proof. Hop $T_{2,1}$ is a simplified version of an operation from the TPC-C benchmark, for which commutativity has not been previously established. Such non-obvious commutative relationships make manual analysis arduous and error-prone, which threatens correctness.

This challenge reveals a need for a system that can automatically and correctly construct a precise SC-graph by distinguishing semantically necessary C-edges from those that are superfluous. To address this, we propose a system that integrates formal verification into SC-graph construction. Our approach models the database state and transaction logic, then translates the commutativity property into a verification query. By leveraging an SMT solver to check this query, we can formally and soundly prove whether a C-edge is required. If the proof succeeds (i.e., the hops commute), the C-edge is pruned. This method systematically eliminates cycles arising from conservative syntactic analysis, permitting a higher degree of transaction interleaving. The primary benefit is an SC-graph that is both *correct*, by removing the risk of human error, and *precise*, by eliminating overly conservative dependencies. In our evaluation, we demonstrate that our system successfully identifies and proves non-trivial commutative operations, which leads to a 1.6× improvement on the scaled TPC-C benchmark.

In summary, this paper makes the following contributions:

- (1) We present a formal modeling for database state and transaction hop semantics, enabling sound, automated commutativity analysis. Our model supports common transactional patterns, including conditional logic and loops.
- (2) We design and implement *VIC3*, a system that automatically analyzes transaction hop code and constructs a precise SC-graph. It integrates our pairwise, sound commutativity checker for transaction hops written in TransAct, our proposed formal language for expressing chopped transactions, to prove and prune commutative C-edges.
- (3) We evaluate *VIC3* in a distributed setup using multiple benchmarks, integrating *VIC3* into the IC3 protocol. Our results show that *VIC3* significantly prunes the SC-graph, improving throughput of the scaled TPC-C benchmark under high contention, compared to other baselines without automated commutativity analysis.

2 BACKGROUND

2.1 Transaction Chopping

Shasha et al. [31] first introduced the concept of transaction chopping. The method chops a transaction into smaller pieces, which we term *hops*, that can be executed independently, thereby improving concurrency and performance. However, this original approach is designed for single-node databases, and its strict requirement on transaction dependencies is often impractical. The core of the method is the SC-graph. The nodes of the graph represent transaction hops, whereas the edges are of two kinds: *sibling edges* (S-edges) that connect hops of the same transaction in their original order, and *conflict edges* (C-edges) that represent conflicts between hops of different transactions. A chopping is valid only if the SC-graph contains no SC-cycles, which are cycles including at least one S-edge and one C-edge. If no such cycles exist, serializability can be guaranteed by applying concurrency control to each hop individually according to the protocol.

We use the example in Figure 2a to explain the process. The system has two transactions: T_1 reduces quantity, and T_2 moves quantity from one to another and restore if the quantity is low. T_1 is a single piece, $T_{1,1}$, whereas T_2 is chopped into two pieces, $T_{2,1}$ and $T_{2,2}$. To construct the SC-graph, we first create two instances for each transaction and we use superscript to distinguish different instances. Then, we must identify conflicts. A conservative static analysis, as used in previous works, creates a C-edge if two pieces access the same table and at least one performs a write. Therefore, a C-edge is drawn from an instance $T_{1,1}^1$ to $T_{2,1}^2$ since they both write to table S . This conservative approach, however, can introduce many C-edges. To reduce the number of edges, the system can use semantic knowledge. For instance, there is no C-edge between two instances of $T_{1,1}$ because the redeem operation is commutative with itself. This case is a *commutative conflict*. The identification of such conflicts must be provided by a developer manual annotation in prior work. The final SC-graph is shown in Figure 2c. In this graph, many SC-cycles exist, one of which is the path $T_{2,1}^1 \rightarrow T_{2,2}^1 \rightarrow T_{2,1}^2 \rightarrow T_{2,2}^2$. Thus, this chopping is invalid. The transaction pieces must be re-chopped to eliminate all SC-cycles to guarantee serializability.

2.2 IC3 Protocol

The IC3 protocol [35] is a distributed transaction protocol that uses the SC-graph to improve system performance and scalability. While the original chopped transaction protocol cannot guarantee serializability if SC-cycles exist, IC3 ensures serializability for

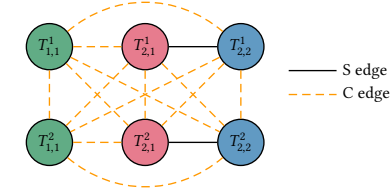
```

transaction T1 (x:int, qty:int)
T1,1 | hop {
    S[ID: x].QTY = S[ID: x].QTY - qty;
}

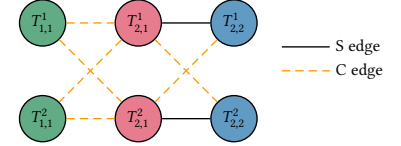
transaction T2 (x:int, y:int, qty:int)
T2,1 | assume qty >= 0 && qty <= 10;
T2,1 | hop {
    if (S[ID: x].QTY - qty >= 10) {
        S[ID: x].QTY = S[ID: x].QTY - qty;
    } else {
        S[ID: x].QTY = S[ID: x].QTY + 91 - qty;
    }
}
T2,2 | hop {
    S[ID: y].QTY = S[ID: y].QTY + qty;
}

```

(a) Example chopped transactions T_1 and T_2 in the TransAct language.



(b) Initial SC-graph based on conservative syntactic analysis is densely connected with C-edges (orange dashed).



(c) A precise SC-graph after pruning commutative C-edges.

Figure 2: A running example of transaction chopping. The initial analysis (b) produces a dense graph with many SC-cycles. After commutative analysis, (c) provides a more precise SC-graph.

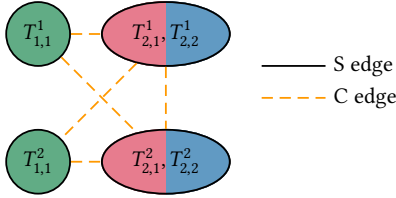


Figure 3: Merged graph after IC3 merges pieces to avoid deadlocks.

any chopping, including those with SC-cycles. The key innovation is a protocol which tracks dependencies and waits among transaction hops to prevent non-serializable executions. However, the protocol benefits from SC-graphs that have fewer SC-cycles, since this reduces the overhead required for dependency tracking. In a distributed system, waits among hops can cause deadlocks across nodes. IC3 avoids such runtime deadlocks by merging deadlock-prone hops into a single atomic hop.

We use our running example to explain this mechanism. Figure 2c is the SC-graph without merging. The SC-cycle $T_{2,1}^1 \rightarrow T_{2,2}^1 \rightarrow T_{2,1}^2 \rightarrow T_{2,2}^2$ is deadlock-prone. To resolve this, IC3 merges the two hops of T_2 into one atomic hop, which eliminates the SC-cycle and prevents the deadlock, as shown in Figure 3. This merge introduces execution overhead; however, it is necessary to guarantee serializability. To further reduce overhead by minimizing SC-cycles, IC3 uses a more precise method to construct C-edges. A C-edge is created only if two hops access the same table and the same field, where at least one hop performs a write operation. IC3 also supports developer annotations for commutative conflicts to reduce C-edges. However, the protocol imposes limitations. It does not consider two write hops to be commutative if one provides a return value used by another. Furthermore, it always defers a commutative hop to the end of its transaction, which reduces flexibility.

3 OVERVIEW

In this section, we provide a formal overview of our verification framework, the architecture of which is illustrated in Figure 4. The purpose of the framework is to automatically verify the SC-graph given the transaction code to make it more precise by formally proving the commutativity of transactional operations.

The process begins with the developer-provided transaction code written in TransAct, which has been previously chopped into hops. Figure 2a shows an example of code written in TransAct. From this code, the compiler framework performs a static analysis of the read and write operations within each hop to construct an initial SC-graph.

The core of our framework is an iterative simplification of this SC-graph. For each C-edge in the graph, we model each hop's operations and the database state. Based on this model, we generate a logical theorem that asserts the commutativity of the two corresponding operations in the Boogie intermediate verification language (Boogie IVL)[3]. The satisfiability modulo theories (SMT) solver, Z3[12], is then employed to decide the validity of *negation* of this verification condition. There are three possible outcomes:

- (1) **Unsatisfiable (UNSAT):** The solver proves that no counterexample to commutativity exists. The property holds, which means the operation is commutative. We then safely remove the C-edge from the graph.
- (2) **Satisfiable (SAT):** The solver finds a specific counterexample that violates commutativity. The conflict is therefore real and must be managed. In this case, the C-edge is kept.
- (3) **Timeout:** The solver cannot determine the result within a predefined time limit. To ensure correctness, we conservatively assume a conflict exists and keep the C-edge.

This verification loop repeats for every C-edge, thus producing a precise SC-graph that deletes C-edges for hops we can prove commute. From this precise SC-graph, the framework then merges hops as necessary to eliminate deadlock-prone SC-cycles, following the IC3 protocol (the merging process needs directed S-edges to

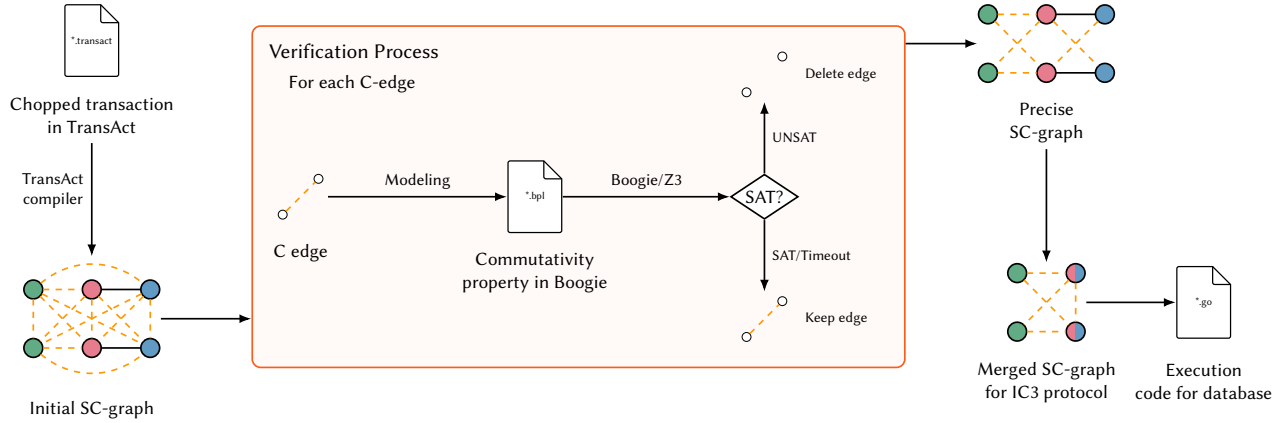


Figure 4: An overview of the verification framework, illustrating the process from static analysis to the generation of optimized execution code.

identify dependencies). Finally, the framework generates the executable code for the database that follows the IC3 protocol to ensure serializability.

We introduce the verification tools in more detail in Section 4, and we present the technical details of modeling and generating verification conditions in Section 5. We discuss the implementation of the database and the performance evaluation in Section 6.

4 AUTOMATED PROGRAM VERIFICATION

To formally reason about transaction semantics, we use techniques from automated program verification. This section provides the necessary background on Satisfiability Modulo Theories (SMT) and the specific toolchain used to automatically generate and solve logical formulas in this work.

4.1 Satisfiability Modulo Theories (SMT)

Our verification approach relies on automated reasoning to prove logical properties. The Boolean Satisfiability (SAT) problem provides the foundation, determining if a propositional formula, a logical formula with Boolean variables and connectives like $\wedge$ (and), $\vee$ (or), and $\neg$ (not), can be made true by some assignment of truth values to its variables.

We use Satisfiability Modulo Theories (SMT), an extension of SAT that reasons about more expressive formulas over a broader set of theories. A theory defines a vocabulary of functions and predicates for a specific domain, along with axioms that constrain their meaning. For instance, the theory of linear integer arithmetic supports reasoning about formulas with variables, integer constants, and operators like $+$, $-$, and $>$. An SMT solver can check the satisfiability of an SMT formula. A possible implementation is to combine a SAT solver’s search for a propositional model with a dedicated theory solver that checks the consistency of domain-specific constraints.

To prove a property is true, we ask the SMT solver to check the satisfiability of its negation. If the solver returns unsatisfiable (UNSAT), it has proven that no counterexample exists, and thus the

original property holds for all possible inputs. If it returns satisfiable (SAT), the solver provides a model, a concrete assignment of values to variables, that serves as a counterexample falsifying the property. This counterexample can be then used for debugging or refining the property.

4.2 Program Verification Toolchain

Although one can manually write an SMT formula to prove a property, this process is often complex. Automated program verification translates a program and its specification into a logical formula, the *verification condition (VC)*, whose validity implies that the program adheres to its specification. Programs introduce state and control flow, which the verification condition must logically encode. To handle state, each update to a program variable creates a new logical variable, transforming an imperative statement like $x = x + 1$ into a logical equality such as $x_1 = x_0 + 1$. Control flow, such as branches and loops, is encoded by systematically exploring execution paths.

The advancement of SMT solvers enables us to prove a complex VC quickly and there are many standard tools developed for VC generation. We use the Boogie intermediate verification language (Boogie IVL)[3] to automate VC generation. Boogie serves as a backend for verifying high-level languages, translating a program represented in Boogie IVL into a VC expressed in the standard SMT-LIB2 format[4]. We use the Z3 theorem prover[12] to solve these VCs. This toolchain relies on standard SMT theories to model program data types and operations, most notably the theory of arrays (ArrayEx), which models memory and collections. The ArrayEx theory defines an array as a map from indices to values, governed by axioms for select and store operations: a select read at an index returns the value stored at that index, and a store at an index updates the value at that index but leaves all others unchanged. Moreover, the axiom of extensionality states that two arrays are equal if and only if they contain the same values at all corresponding indices. This axiom defines array equality based on contents rather than on identity or history.

$$\forall A, B : (\forall i : \text{select}(A, i) = \text{select}(B, i)) \implies A = B$$

As an illustration, consider proving the commutativity of two transaction hops $T_{1,1}, T_{2,1}$ (denoted as h_1, h_2) from our running example in Figure 2a, that operate on a shared array S . Commutativity requires that executing h_1 then h_2 results in the same final state as executing h_2 then h_1 . The verification toolchain automatically constructs a VC for this property. It models the two execution paths, creating logical representations of the array state after each sequence of operations ($A_{1,2}$ and $A_{2,1}$). The array state depends on initial array (A_0) and parameters (x_1, q_1, x_2, y_2, q_2). For example, the array $A_{1,2}$ can be written as

$$A_{1,2} = \text{if } (a - q_2 \geq 10) \text{ then store}(A_1, x_2, a - q_2) \\ \text{else store}(A_1, x_2, a + 91 - q_2)$$

where

$$A_1 = \text{store}(A_0, x_1, \text{select}(A_0, x_1) - q_1) \\ a = \text{select}(A_1, x_2)$$

The array $A_{2,1}$ can also be expressed as an expression in term of A_0 and parameters. Then, the final VC passed to the SMT solver is a universally quantified theorem asserting their equality for all possible initial states and parameters:

$$\forall A_0, \text{ valid params. } A_{1,2} = A_{2,1}$$

If the SMT solver finds this formula to be valid, the hops are proven to be commutative.

4.3 Soundness and Completeness

Our framework reasons about an abstract *model* of the real system. The correctness of this approach depends on two properties: soundness and completeness.

A system is *sound* if any property it proves is true in the actual system; our framework therefore guarantees that if it proves two hops commute, they are in fact commutative. Soundness is a mandatory property for our purposes.

A system is *complete* if it can prove every true property. A complete system would thus prove commutativity for all operations that are genuinely commutative.

Fundamental work on computability [28, 34] has shown it to be impossible to achieve both soundness and completeness in this context. *VIC3* is thus designed to be conservative: it defaults to declaring a conflict unless it can mathematically prove otherwise. This amounts to partially sacrificing completeness to achieve soundness.

5 FORMAL MODELING OF DATABASE

To reason about the commutativity of transaction hops, we require a precise mathematical model of the database state and the operational semantics of hops that transform it. We represent the system using the SMT-based framework introduced in Section 4.1. This representation abstracts from implementation details and focuses on the logical properties of state and its transformations.

5.1 Modeling Transactions, Hops, and State

This section formalizes the core concepts of our transaction execution model, including transaction templates, instances, hops, and their local state.

Definition 5.1 (Transaction Template). A *transaction template*, denoted T_i , is a statically defined program. At runtime, multiple instances of a given template may be executed.

Transaction templates serve as blueprints for execution. For example, a template might define the logic for a "purchase item" operation, which is instantiated with different item IDs and customer accounts each time it runs. Our analysis reasons about these templates statically and their concrete executions at runtime.

Definition 5.2 (Hop). Each transaction template T_i is decomposed into a finite sequence of *hops*, which are the indivisible units of execution in our model. The j -th hop of transaction template T_i is denoted by $T_{i,j}$. When the specific template or index is not relevant, we use h to denote an arbitrary hop.

Definition 5.3 (Transaction and Hop Instances). A *transaction instance*, denoted T_i^k , is the k -th runtime execution of the transaction template T_i . Consequently, a *hop instance*, denoted $T_{i,j}^k$, is the execution of hop $T_{i,j}$ within the context of transaction instance T_i^k .

Definition 5.4 (Local State of a Hop Instance). The *local state* of a hop instance h , denoted ρ_h , is the environment mapping local variable identifiers to their values at the beginning of that instance's execution. This state contains:

- the input parameters of the enclosing transaction instance,
- any live variables propagated from preceding hops.

5.2 Modeling Database State as Nested Arrays

We model the database state by representing each non-primary-key column of a table as a distinct logical array. The semantics of these arrays are defined by the ArrayEx theory introduced in Section 4.2. For a table with a composite primary key consisting of n components, the corresponding column is modeled as an array nested n levels deep. A table with a single primary key is the base case where $n = 1$. For example, consider a column c in a table whose primary key is the pair (k_1, k_2) . The state of this column, A_c , is represented as a nested array. Accessing the value for this composite key requires a nested sequence of select operations: $\text{select}(\text{select}(A_c, k_1), k_2)$.

This nested array model directly inherits the axioms of the underlying ArrayEx theory. The essential properties of single-dimensional arrays therefore extend to these multi-dimensional structures. First, a select at a composite key returns the value most recently written to that key by a store. Second, a store at one composite key does not alter the value at any other distinct composite key. Third, two nested arrays representing the same column are equal if and only if they return the same value for every possible composite key. These properties allow the SMT solver to reason soundly about state transformations on complex table structures.

Our verification approach operates over a symbolic database state rather than a single concrete instance. This symbolic state can be viewed as an abstract table in which the value for any primary key is an unconstrained logical variable. Such a variable is not fixed

to a specific constant (e.g., “100” or “John Doe”); it can denote any value of its type. Consequently, any property we prove holds for all possible initial database states.

The initial symbolic state is constrained by integrity constraints, which characterize valid database states. For example, a column balance may be required to always contain a non-negative value. We encode this requirement as a logical axiom that the symbolic state must satisfy: $\forall k : \text{select}(A_{\text{balance}}, k) \geq 0$. By incorporating global data invariants, the verification can leverage a more precise characterization of the database state, increasing opportunities to prove commutativity.

5.3 Modeling Data Types and Functions

To verify transaction logic, we must translate not only the database state, but also the data types and functions used in the transaction code into the SMT framework. In this framework, all functions are modeled as pure mathematical functions: they are stateless and deterministic, and for a given set of inputs they always produce the same output without side effects. This design facilitates logical reasoning, but requires specific techniques to represent complex data structures and inherently stateful operations from imperative code.

We model complex data types, such as strings or lists, as uninterpreted sorts whose behavior is specified axiomatically. An uninterpreted sort is a type declared to the solver without any internal structure; its operational properties are instead characterized by logical axioms. For example, to model a generic list, we introduce an uninterpreted sort `List` and declare associated pure functions such as `append` and `length`. We then provide axioms governing their behavior:

$$\begin{aligned} \forall l, e : \text{length}(\text{append}(l, e)) &= \text{length}(l) + 1 \\ \forall l, e : \text{get}(\text{append}(l, e), \text{length}(l)) &= e. \end{aligned}$$

These axioms give the SMT solver sufficient information to reason about list operations. However, this abstraction is also a source of incompleteness. For example, the theory of string operations is undecidable [11, 26], so any practical, automated verification must rely on a simplified model. Our framework models each string literal as a distinct symbolic value, which may ignore semantic equivalences that could be crucial for proving commutativity, thus trading completeness for decidability and efficiency.

This axiomatic approach works well for stateless data types but does not directly apply to inherently stateful functions. A typical example is `genUUID()`, which is intended to return a fresh identifier on each call. Modeling `genUUID()` as a stateless pure function would incorrectly imply that all calls return the same value, contradicting its intended uniqueness property.

To correctly model such stateful functions, which we denote as f_{stateful} , we transform them into pure functions f_{pure} by making the call context an explicit argument. The context must uniquely identify every invocation of the function across all possible executions of transaction hops. We represent this context as a tuple (n, m) and tie it to the hop notation introduced above:

- The first parameter, n , is the *call-site index*. It is a static, zero-based integer that identifies the lexical location of the function call within the source code of a hop $T_{i,j}$. For a

given hop, distinct call sites of the same stateful function are numbered $n = 0, 1, \dots$ based on their static positions.

- The second parameter, m , is the *hop instance identifier*. For hop instance $T_{i,j}^k$, the identifier is the instance number, hence, $m = k$.

This pairing (n, m) therefore uniquely identifies each concrete invocation of f_{stateful} and yields a corresponding pure call $f_{\text{pure}}(n, m)$.

For the stateful function `genUUID()`, we enforce uniqueness by adding an injectivity axiom stating that distinct call contexts yield distinct results:

$$\begin{aligned} \forall n_1, m_1, n_2, m_2 : (n_1 \neq n_2 \vee m_1 \neq m_2) \\ \implies \text{genUUID}_{\text{pure}}(n_1, m_1) \neq \text{genUUID}_{\text{pure}}(n_2, m_2). \end{aligned}$$

This axiomatization allows the SMT solver to reason about the outputs of stateful operations within the purely functional SMT paradigm while preserving their essential properties.

5.4 Operational Semantics of Hops

We model the execution of a hop h as a state transformation from an initial configuration $\langle p, S \rangle$. The global state S is the set of symbolic nested arrays representing the database columns. The local state p is a mapping from variable identifiers to values, containing the hop’s input parameters and any variables passed from preceding hops in the same transaction. The values in both S and p are unconstrained logical variables, so the initial configuration represents all possible valid initial states.

The semantics of hop h are given by a function τ_h that maps the initial configuration to a final configuration $\langle p', S' \rangle$:

$$\tau_h : \langle p, S \rangle \rightarrow \langle p', S' \rangle.$$

This function is not an interpreter that executes the code step by step. Instead, it denotes the overall mathematical function from input state to output state. Its purpose is to generate a set of logical constraints relating $\langle p, S \rangle$ and $\langle p', S' \rangle$. A verification-condition (VC) generator implements this translation by analyzing the hop’s implementation, including its primitive statements and control flow, and producing a single logical formula that characterizes the hop’s effect.

The body of a hop consists of a sequence of primitive statements. The semantics of each statement type are defined by the models introduced above. Database statements, such as reads and writes, are modeled as applications of `select` and `store` on the nested-array representation of the database state. All other statements involving local computation, from arithmetic to stateful operations such as unique-identifier generation, are modeled as calls to pure functions, including the context-parameterized encodings of stateful functions from Section 5.3. Thus, every statement within a hop has a direct translation into the logical framework.

The control flow of a hop is translated into a logical formula by the VC generator. As discussed in Section 4.2, a VC generator translates a code fragment, including its control-flow constructs, into a single formula that is valid if and only if the code satisfies its specification. A conditional branch is encoded using implications that capture the effects of each branch, so that the resulting formula represents all possible execution paths simultaneously. The

translation of loops is a non-trivial case, which we address in detail in Section 5.6.

In summary, τ_h captures the denotational meaning of a hop. It aggregates the logical constraints induced by all primitive statements and the hop's control flow. This function maps the unconstrained input state to a constrained output state, yielding a precise logical specification of both the final global database state S' and the final local state p' . The local state may change shape: intermediate variables can be discarded and new variables, to be consumed by subsequent hops, can be introduced. The resulting logical formula provides a formal specification of the hop's behavior that can be checked by an SMT solver.

5.5 Commutativity and Proof Methodology

We now formalize commutativity between two hops h_A and h_B . The complete system configuration is represented by a tuple $\langle p_A, p_B, S \rangle$, where S is the shared global database state and p_A and p_B are the local states of each hop. The behavior of each hop is defined by a transformation function, τ_A or τ_B , as in Section 5.4.

We establish commutativity using two-path symbolic execution. Starting from a fully general symbolic initial state, we symbolically compute the final system configurations for both execution orders, h_A followed by h_B and h_B followed by h_A . Commutativity is then expressed by asserting that these two final configurations are logically equivalent. This reduces the proof over an infinite family of concrete states to the verification of a single logical formula.

Definition 5.5 (Commutativity). Two hops h_A and h_B commute if, for any initial configuration, the final system configuration is identical regardless of execution order. Formally:

$$\begin{aligned} \forall p_{A,0}, p_{B,0}, S_0 : & \left(\langle p_{A,A}, S_A \rangle = \tau_A(p_{A,0}, S_0) \right. \\ & \wedge \langle p_{B,AB}, S_{AB} \rangle = \tau_B(p_{B,0}, S_A) \\ & \wedge \langle p_{B,B}, S_B \rangle = \tau_B(p_{B,0}, S_0) \\ & \left. \wedge \langle p_{A,BA}, S_{BA} \rangle = \tau_A(p_{A,0}, S_B) \right) \\ \Rightarrow & \left(p_{A,A} = p_{A,BA} \wedge p_{B,AB} = p_{B,B} \wedge S_{AB} = S_{BA} \right). \end{aligned}$$

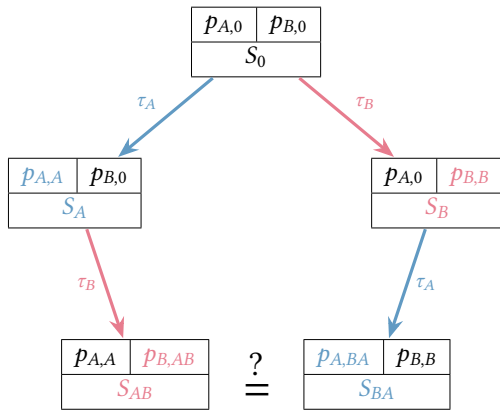


Figure 5: Two-path symbolic execution used to prove the commutativity of hops h_A and h_B .

Figure 5 visually illustrates the above formula. The tuple $\langle p_A, p_B, S \rangle$ is depicted as a table. The process starts from the universal initial state $\langle p_{A,0}, p_{B,0}, S_0 \rangle$ at the top. The diagram then branches into two execution orders. The left path applies τ_A first, followed by τ_B ; the right path applies τ_B first, followed by τ_A . Each transformation updates specific components of the system state, highlighted in the corresponding color. For example, the blue arrow for τ_A leads to a state where the initial local state $p_{A,0}$ has been updated to $p_{A,A}$ and the global state S_0 has been changed to S_A .

The implication $\Rightarrow$ in the definition is represented by the $\stackrel{?}{=}$ symbol at the bottom of the figure. The formula states that, for all initial values of $p_{A,0}$, $p_{B,0}$, and S_0 , the final states must coincide. To prove commutativity, we must therefore show equality of the final local states for hop A ($p_{A,A} = p_{A,BA}$), the final local states for hop B ($p_{B,AB} = p_{B,B}$), and the final global database states ($S_{AB} = S_{BA}$). If these equalities hold, then the execution order of h_A and h_B does not affect the final outcome.

The correctness of this methodology follows from the nature of symbolic execution. The commutativity formula is submitted to an SMT solver, which searches for a counterexample: an assignment of concrete values to the initial symbolic variables $p_{A,0}$, $p_{B,0}$, and S_0 that violates the final equalities. If no counterexample exists, the solver proves that the equalities hold for all possible initial values. Thus, we verify commutativity for every concrete initial configuration of the system. Also, this is a source of incompleteness. By verifying hops (h_A, h_B) in a pairwise, isolated manner, the initial state $\langle p_{A,0}, p_{B,0}, S_0 \rangle$ is unconstrained beyond global integrity constraints. This loses contextual information that would have been established by prior hops within their respective transactions.

5.6 Modeling of Loops

Control flow with loops poses a fundamental challenge for automated program verification. Because a loop can traverse an unbounded number of program states, complete analysis is generally impossible due to the undecidability of the halting problem. Verification frameworks therefore rely on abstraction techniques to reason about loops finitely. Our approach distinguishes between two categories of loops commonly found in database transactions and applies a dedicated modeling strategy to each.

Bounded Loops. The first category consists of loops that iterate over a statically known, finite range. The number of iterations is typically determined in the transaction code (e.g., iterating over a list of five items to be inserted into a shopping cart). For this class of loops, our system employs full *loop unrolling*: the loop is expanded into a finite sequence of straight-line code, with one code block per iteration. Because the number of iterations is fixed and known, this transformation provides an exact representation of the loop's semantics and does not compromise the soundness or completeness of our verification.

Unbounded Loops. The second category consists of loops whose iteration count is dependent on the database state, making it impossible to fully unroll at compile time. The canonical example is a SELECT query that scans a table. The number of rows returned by such a query is not known statically, as it depends on the current

contents of the database. While one could bound the loop at a pre-defined maximum table size, this is impractical as it would create a prohibitively large number of paths for an SMT solver to check.

Instead, we model such a loop via a single, symbolic iteration: the loop body is analyzed once, using a set of unconstrained primary keys to represent an arbitrary row from the database. These keys are not bound to any concrete values; thus, they can represent the data from any arbitrary row the loop could possibly process.

This approach is sound: a simple proof by induction shows that if a hop commutes with every loop iteration, it commutes with the loop. Consequently, any data-dependent conflict will be identified. However, this abstraction is a source of incompleteness. We create a sound over-approximation of the loop’s behavior, which prevents the verifier from using other behaviors of the loop that could establish commutativity. For example, if one loop iterates over a table to sum values and another hop updates values without affecting the sum, the verifier cannot prove commutativity because it cannot reason about the loop’s accumulation behavior.

6 EVALUATION

In this section, we evaluate the performance of *VIC3* with three different benchmarks. We compare with the traditional 2PC distributed transaction protocol and the original IC3 protocol, which is not optimized by our verification system. We also measure the overhead of the verification and explore the characteristics of commutativity in the three benchmarks.

6.1 Implementation and Setup

TransAct. We built a programming language, TransAct, and a compiler to analyze and generate executable code for database transactions. *VIC3* contains a front-end parser, dataflow analysis, SC-graph construction, commutativity verification, and code generator that targets the Go language. Our compiler automatically generates the commutativity verification conditions in Boogie IVL for each C-edge in the SC-graph and then updates the SC-graph according to the results from the Z3 SMT solver.

Distributed IC3 Database. We built a distributed IC3 database to evaluate the performance of the generated transaction chopping code. The prototype is implemented in Go, with BoltDB [18] for persistent key/value storage. Each transaction is managed by a coordinator, who dispatches the hops to the corresponding server one by one. Each hop runs as a BoltDB transaction, and the hop’s code is generated by the TransAct compiler. The prototype executes merged hops using the two-phase commit protocol with wound-wait locking to ensure atomicity and consistency.

Comparison Baselines. In the evaluation, we compare *VIC3* with the original IC3 protocol and two-phase commit protocol (2PC). Both *VIC3* and IC3 are evaluated using the distributed IC3 database described above, and they have the same generated hop processing code. The only difference is the generated SC-graph, where commutative edges in *VIC3* are removed by the verifier. For 2PC, we implemented it in the distributed IC3 database to execute merged hops. The same implementation is used to evaluate 2PC by executing the whole transaction as a single merged hop.

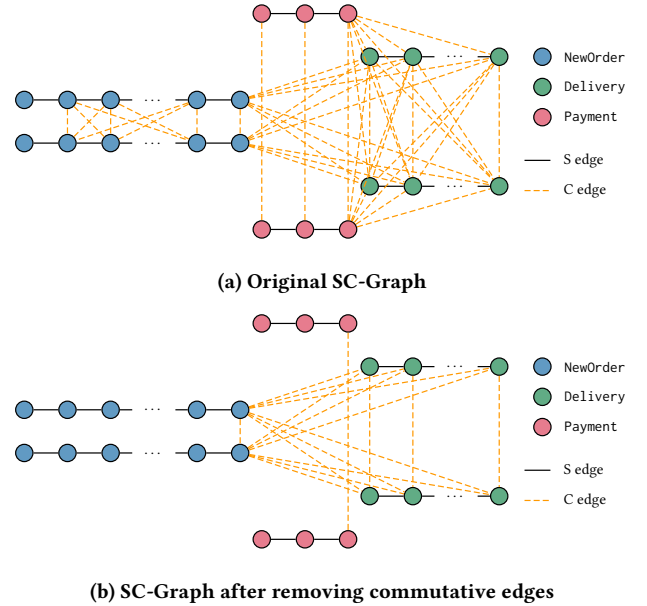


Figure 6: SC-Graph of the scaled TPC-C benchmark

Benchmarks. We evaluate the performance of *VIC3* using three benchmarks: TPC-C, RUBIS, and SOCIAL. TPC-C [9] is a wholesale supplier application commonly used in evaluating the performance of online transaction processing (OLTP) workloads. The latter two benchmarks are used to evaluate the performance of transaction chopping in prior work [39]. RUBIS [1] is an auction service web application where users can sell, browse, and place bids on items. SOCIAL benchmark includes basic operations for social networking applications, allowing users to add friends and post their status updates.

Experiment Setup. The experiments are conducted on Cascade Lake R nodes of Chameleon Cloud [7]. Each machine has 2 Intel Xeon Gold 6240R CPUs at 2.4GHz, each with 48 virtual threads. It is also equipped with 192 GB RAM and 10 Gigabit Ethernet.

Unless otherwise stated, we run four Docker containers to represent data centers in different geographic areas. Each container runs 10 server processes, resulting in a total of 40 database servers. The network round-trip time (RTT) for servers in different Docker containers is 40ms (approximately half of the AWS us-west-1 to us-east-1 RTT). There is a single client that generates transaction requests with a thread pool (64 worker threads by default). For each request, a worker thread sends it to the corresponding database server that hosts the data accessed by the first hop of the request and waits for the transaction to commit.

6.2 TPC-C

Scaled TPC-C Description. We scaled the TPC-C benchmark by partitioning tables at a finer-grained level, allowing more parallelism in the distributed setting. Each table is partitioned at the district level (for the District, Customer, Order, ... tables) or item level (for the Item and Stock tables) instead of the warehouse. For

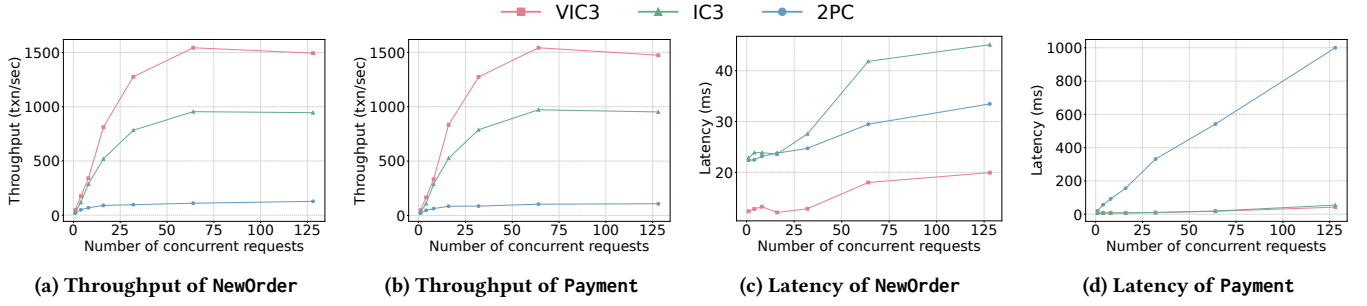


Figure 7: Performance of the TPC-C benchmark

example, a customer with key (W_ID, D_ID, C_ID) is partitioned to the server $W_ID \times 10 + D_ID$, and a stock with key (W_ID, I_ID) is partitioned to server $W_ID \times 10 + I_ID \% 10$. The item table is replicated to each warehouse so that each stock is co-located with the corresponding item. In our experiment setup, each warehouse is managed by a Docker container, and each district is hosted by one of the servers within the container.

Figure 6 shows the SC-graph of the three read-write transactions. The NewOrder transaction is chopped into 12 hops: the first hop reads the warehouse, followed by 10 hops that update the stock of each item in the order, and the last hop that updates the district and inserts order tables. In the original SC-graph, all item hops conflict with each other due to the update of the stock quantity. Figure 2a shows the code in the item hops that leads to the conflict. This results in merging all item hops. However, our verifier proves that the operations on the stock quantity are commutative, given that the quantity of each orderline is 1-10. After removing the C-edges between the item hops, the NewOrder transaction contains no SC-cycle and no merged hop, which can significantly improve the performance as shown in the evaluation result.

The Payment transaction is chopped into 3 hops. The first two hops increase the revenue of the warehouse and the district, respectively. The last hop updates the customer information. The C-edge between the warehouse/district hops of two Payment instances can be identified as commutative trivially because of the increment operation. The Delivery transaction is chopped into 10 hops, where each hop delivers one order in a district. The district hops are conflicting with each other, and those C-edges are not commutative. They form a directed SC-cycle, leading to merging the whole Delivery transaction into one single merged hop.

Evaluation. We run TPC-C with a mixed workload of the three read-write transactions and vary the number of concurrent requests to evaluate the performance under different system loads. We deployed 4 warehouses in this experiment, which is considered very high contention for TPC-C. We report the result of the NewOrder and Payment transactions, each represent 48% of the workload.

Figure 7a and Figure 7b shows the throughput of NewOrder and Payment transactions, respectively. They have the same throughput because of the equal distribution. VIC3 scales almost linearly under a light workload. It achieves the peak throughput of 1544 txn/sec for the NewOrder transaction at 64 concurrent requests. Under a heavy workload, the throughput of VIC3 saturates. This is because

the CPU and I/O reach their maximum capacity, and the processing of a hop becomes the bottleneck instead of the concurrency control overhead. IC3 shows similar behavior under different system loads, but it has significantly lower performance (954 txn/sec maximum) compared to VIC3. In the original SC-graph used by IC3, there is a merged hop involving 83% of hops in the NewOrder transaction. As a result, the NewOrder transaction for IC3 is mostly executed using 2PC, which requires extra message round-trips compared to VIC3. 2PC experiences very low throughput because there are only 4 warehouses, which leads to very high contention. For VIC3 and IC3, the warehouse locks are held for a short period during local hop processing, so the contention level of warehouses does not significantly affect the performance. However, 2PC requires Payment to hold an exclusive warehouse lock for the entire transaction duration. Therefore, the limited number of warehouses becomes the primary bottleneck of 2PC.

Figure 7c and Figure 7d show the latency of NewOrder and Payment transactions. For NewOrder, all protocols have stable latency under a light workload, and start to increase under a heavy workload. As explained above, VIC3 has a much lower latency due to the removal of the merged hop. IC3 and 2PC have similar results because IC3 executes the NewOrder transaction mostly using 2PC, due to the dominating size of the merged hop. After 32 concurrent requests, IC3 has a higher latency compared to 2PC. This is because IC3 has a much higher throughput than 2PC that stresses the computing resources. IC3 is also more sensitive to the increment of the workload, compared to VIC3, due to the overhead of dependency tracking to enforce the correct execution order of the undirected SC-cycle. The latency of the Payment transaction shows the bottleneck of 2PC: Unlike NewOrder that acquires a shared read lock on the warehouse table, Payment acquires an exclusive write lock. There can be at most 4 concurrent Payment transactions, and most of the Payment transactions are being blocked when the number of concurrent requests is high.

6.3 RUBIS

Benchmark Description. The original implementation of RUBIS used a single database server, and we modified it to a distributed application. The modified benchmark contains 6 tables and 14 transactions. There are 8 read-only transactions, where 6 of them contain

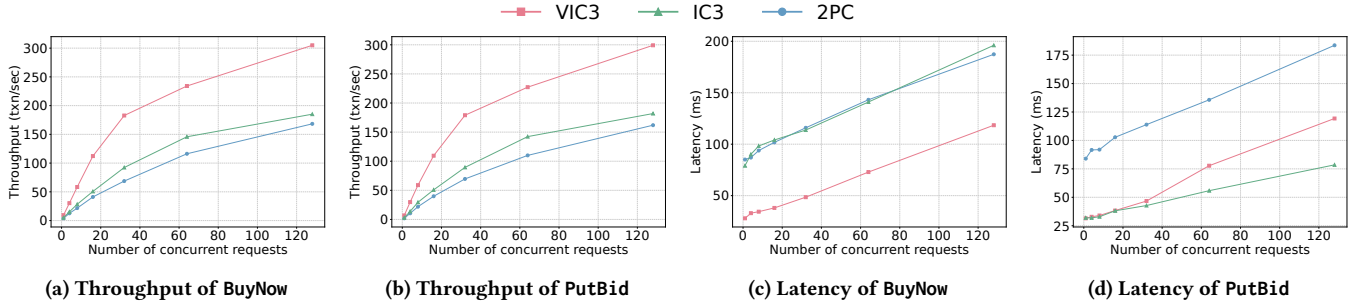


Figure 8: Performance of RUBIS benchmark

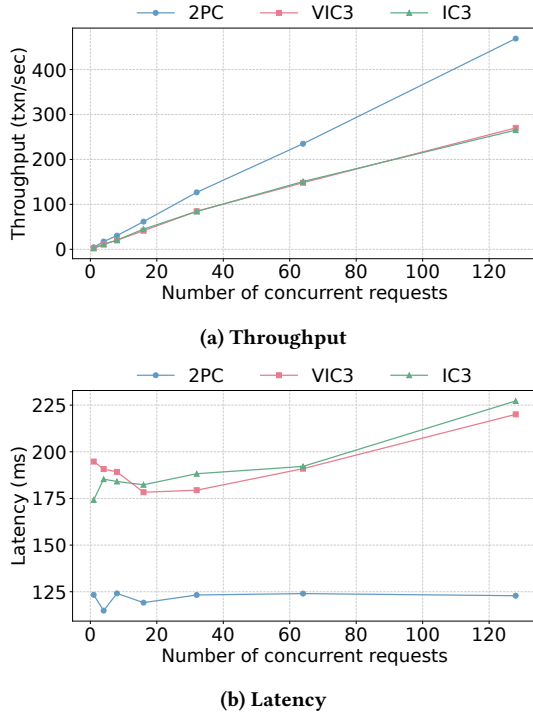


Figure 9: Performance of the BeFriend transaction in the SOCIAL benchmark

full table scans that require accessing all partitions. In the experiment, we report the performance of PutBid and BuyNow transactions, which involve 50% of the transaction workload. PutBid is a two-hop transaction where the first hop updates an item if the bid value is higher than the current high bid of the item, and the second hop inserts the new bid into the Bid table. There is an undirected SC-cycle between the PutBid transaction and two other transactions that read the Bid table. BuyNow is also a two-hop transaction, but it is involved in a directed SC-cycle and needs to be merged if the commutative edges are not identified.

Evaluation. We run RUBIS with a mixed workload of all transactions and vary the number of concurrent requests. We report

the result of the BuyNow and PutBid transactions, both of which represent 25% of the whole workload.

Figure 8a and Figure 8b show the throughput of the BuyNow transaction and the PutBid transaction. The throughput results are the same for both transactions because they have equal distribution in the workload. VIC3 significantly outperforms IC3 because it removes two merged hops with the commutativity verifier. The merged hops are expensive in the distributed setting due to the need for multiple round-trip coordination. For IC3, it outperforms 2PC by holding locks for a shorter period and allowing more interleaving of the distributed transactions.

Figure 8c and Figure 8d show the latency of the two transactions, respectively. The BuyNow transaction consists of two hops that need to be merged without verification. IC3 and 2PC have the same latency because IC3 executes BuyNow entirely using 2PC. VIC3 has a much lower latency since it removes the commutative edges and can run the transaction without the multiple message round-trips. The PutBid transaction consists of two hops that form an undirected SC-cycle, which does not need to be merged. Both VIC3 and IC3 have lower latency compared to 2PC. However, under a heavy workload with the number of concurrent requests larger than 64 threads, VIC3 has a higher latency than IC3 due to the saturated computing resource: while both protocols execute the PutBid transactions in the same way, VIC3 has about 1.7× higher total throughput compared to IC3, leading to more CPU-intensive workloads.

6.4 SOCIAL

Benchmark Description. The SOCIAL benchmark includes basic operations for social networking applications. It consists of 5 tables and 6 transactions, where 2 of them are read-only transactions. In the experiment, we report the performance of the BeFriend transaction (25% of the workload), which is chopped into 4 hops. The first two hops insert $(user1, user2)$ and $(user2, user1)$ tuples to the Graph table, representing the friend relationship between user1 and user2. These two hops form a directed cycle with the ReadFriends transaction, so they need to be merged. The last two hops insert two BeFriend logs into the Activity table. They also conflict with the ReadActivities transaction, which results in another merged hop in BeFriend. The conflicts between BeFriend and the two read-only transactions are strict read-write conflicts

Table 1: Summary of commutativity verification results for each benchmark.

Benchmark	Total C-edges	Commutative	Non-Commutative	Commutative Rate	Total Time (s)
TPC-C	284	242	52	85.2%	106.6
RUBIS	91	12	79	13.2%	3.9
SOCIAL	128	10	118	7.8%	4.7

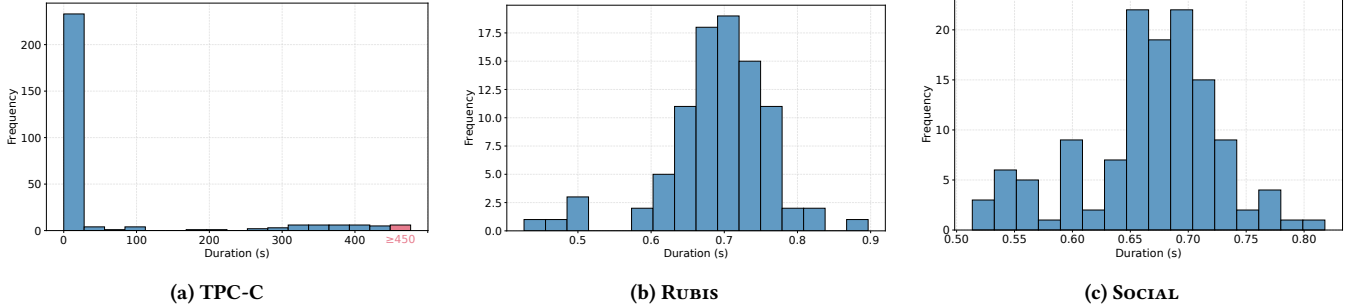


Figure 10: Histograms of verification times for C-edges in the TPC-C, RUBIS, and SOCIAL benchmarks. The time taken for the majority of verification checks is less than 1 second, although a small number of complex cases require more time.

that are not commutative. Thus, the two merged hops cannot be removed after using our commutativity verifier. We use this example to illustrate the limitations of IC3 and VIC3 for certain transaction types.

Evaluation. We run SOCIAL with a mixed workload of the six transactions and vary the number of concurrent requests. We report the result of the BeFriend transaction.

Figure 9a and Figure 9b show the throughput and latency of the BeFriend transaction. VIC3 and IC3 have the same performance on both throughput and latency, because only 7.8% of the C-edges can be identified as commutative with the verifier, as shown in Table 1. Most of the identified commutative edges are not the primary source of the data contention, so they cannot help improve the performance. Due to the two merged hops, VIC3 and IC3 have to run 2PC twice, which hurts the performance significantly. 2PC outperforms VIC3 and IC3 because the multiple round-trip coordination occurs only once.

The result shows that in a distributed environment, the IC3 protocol may have poor performance when there are multiple merged hops within a single transaction. VIC3 cannot mitigate the impact of merged hops if there are very few commutative edges in the application.

6.5 Verification Time

We evaluate the verification overhead of the system on the three benchmarks. All the experiments are conducted on a different machine with Intel i7-12700K up to 5.0GHz and 128 GB RAM with Boogie version 3.5.5.0 and Z3 version 4.8.12. The Boogie verifier is configured to verify each C-edge in a single thread with a loop unrolling bound of 12 and a timeout of 30 seconds. The reason for choosing a different machine is to simulate the scenario where the verification is performed on a developer’s machine before deploying the generated code to the distributed database cluster.

We verify each C-edge for commutativity. Table 1 summarizes the results of this verification process. The commutative rate is the fraction of C-edges proved commutative. We empirically confirm that the results of SC-graph with this 30-second timeout are identical to those obtained without any timeout, which is what we use in our evaluation. The TPC-C benchmark exhibits a high pass rate, whereas the pass rates for RUBIS and SOCIAL are considerably lower. This is because many operations in TPC-C, such as updating stock levels, are naturally commutative.

To analyze the performance of the verifier itself, we measure the time required to verify each C-edge. Figure 10 presents histograms of the verification times for each of the three benchmarks. The x-axis represents the verification time in seconds, and the y-axis shows the number of C-edges that fall into each time bucket.

As shown in the figures, the verification time for most C-edges is under one second for all benchmarks. However, there exist complex cases in the TPC-C benchmark, where verification can take longer due to more intricate data dependencies and loop in transaction logic. The edges between hops of transaction Delivery and one hop of transaction NewOrder take approximately 5 minutes to verify, as each has a for-loop that iterates through tables. There is one C edge between two hops of transaction NewOrder that takes a few hours to verify, similarly because of the for-loop in both hops. These results demonstrate that the verification process is efficient for the vast majority of pairs, making it a practical approach for automated commutativity analysis.

7 RELATED WORKS

In this section, we review the related literature in three primary areas: concurrency control, commutativity analysis, and formal verification.

Static Analysis of Transactions. Using transaction semantics to improve concurrency has been explored extensively in prior

work [2, 16, 29]. Deterministic database systems perform static analysis of transactions to improve concurrency by applying rules to a batch of transactions and enforcing concurrency control at the batch level rather than for individual transactions [10, 14, 25]. These systems ensure serializability while minimizing transaction conflicts. For example, Calvin [33] and BOHM [13] use knowledge of transaction conflicts to relax a total order into an equivalent partial order with higher concurrency. Similar ideas appear in other deterministic databases. However, our system applies semantic analysis to *transaction types* rather than batches of predefined transactions, which makes its scope distinct from prior approaches. Moreover, transactions execute individually with reduced conflicts, without requiring runtime analysis over batches.

Transaction chopping protocols [17, 24, 31, 36, 38, 39] aim to decompose transactions into smaller subtransactions while preserving serializability. However, these methods are often too rigid, and their conservative analysis yields an SC-graph with many cycles, which restricts concurrency and limits applicability. Authors in [6] study transaction chopping under Parallel Snapshot Isolation (PSI). Rococo [24] introduces deferrable hops and reorders those hops during runtime to tolerate certain types of SC-cycles. IC3 [36] relaxes the strictness of transaction chopping by allowing SC-cycles and combining static analysis with dependency tracking. Brook2PL [17] further increases flexibility by enabling partial chopping and allowing dependencies among subtransactions. However, these methods still inherit a fundamental limitation of transaction chopping protocols. Their performance is bounded by the precision of the SC-graph, since SC-graphs often contain many unnecessary C-edges that make the analysis overly conservative. They rely on the developers to manually annotate commutative edges, which is time-consuming and error-prone for large applications. VIC3 directly addresses this gap by automatically constructing a precise SC-graph using formal verification to identify and prune semantically unnecessary C-edges.

Commutativity Analysis. Using commutativity to improve concurrency has a long history in concurrent systems [8, 20, 22, 23, 37]. Early work by Weihl [37] defines atomic data types and shows how commutativity can be incorporated into two-phase locking protocols. Reduction [23] simplifies proofs of parallel programs by identifying operations, called movers, that can be reordered without changing the program’s observable behavior. This notion of safe reordering is closely related to commutativity. More recently, Mover Logic [15] builds on reduction to develop a formal program logic for verifying multithreaded software. Authors in [8] explore a scalable commutativity rule by developing Commuter, a tool that uses symbolic execution to automatically identify conditions under which operations commute, enabling the discovery of scalability bottlenecks in operating system kernels. Authors in [30] explore commutativity in the context of distributed systems, proposing conflict-free replicated data types (CRDTs) that leverage commutative operations to ensure eventual consistency without coordination.

These works highlight the importance of commutativity for improving concurrency and propose different techniques to analyze and exploit it. However, they rely on substantial manual annotations, and the implementation of their commutativity analysis is

complex. In contrast, VIC3 automatically *verifies* commutativity, rather than using reduction to find conditions for commutativity. VIC3 constructs a formally verified and precise SC-graph for database transactions without extensive manual effort. In addition, our approach is tailored to transaction chopping, where each hop is atomic, while prior works assume instructions as the atomic unit.

Formal Verification. VIC3 leverages automated formal verification to prove the commutativity of hops. Our verification pipeline employs a standard software verification architecture, using the Boogie intermediate verification language [3] to generate verification conditions and the Z3 SMT solver [12] as the backend prover.

While our foundational tools are standard, their application is novel. High-level verification frameworks such as Dafny [21] and SMACK [27] use a similar architecture to prove the correctness of entire programs. Likewise, interactive proof assistants like Rocq Prover [32] (formally known as Coq) have been used to formally verify complex systems, including the C language standard [19] and embedded systems [5]. However, these tools focus on proving high-level, developer-specified properties and typically require significant manual effort.

VIC3 pioneers a different approach by applying automated verification to a fine-grained, domain-specific problem: proving the commutativity of arbitrary code fragments to optimize transaction schedules. By bridging automated formal methods with database transaction theory, our work creates a system that is both provably correct and highly automated.

8 CONCLUSION

In this paper, we addressed the challenge of building correct and precise SC-graphs for transaction chopping. While transaction chopping offers a path to higher concurrency, its performance is fundamentally limited by the precision of its dependency analysis. The manual identification of commutative operations to optimize the SC-graph is not only arduous but also susceptible to human error, which can compromise system correctness.

We have presented a novel framework that automates and formalizes this critical process. By modeling transaction semantics and database state within a formal language, we can automatically translate the complex property of commutativity into a verification problem solvable by an SMT solver. Our system systematically analyzes each potential conflict, formally proving whether it is semantically necessary or can be safely removed. Although the analysis is incomplete, this automated approach guarantees the construction of an SC-graph that is both correct and precise, eliminating the risks of manual analysis and uncovering non-obvious opportunities for optimization.

Our evaluation demonstrates the practical benefits of this method, showing significant performance gains by enabling more transaction interleaving under the IC3 protocol. The verification process, while computationally intensive for some complex cases, is efficient for the majority of operations, making it a viable tool for developing high-performance distributed systems. By bridging the gap between formal methods and database concurrency control, this work provides a sound and automated foundation for building faster and more reliable distributed applications.

REFERENCES

- [1] Amza, Chanda, Cox, Elnikety, Gil, Rajamani, Zwaenepoel, Cecchet, and Marguerite. 2002. Specification and implementation of dynamic web site benchmarks. In *2002 IEEE international workshop on workload characterization*. IEEE, 3–13.
- [2] Peter Bailis, Alan Fekete, Michael J Franklin, Ali Ghodsi, Joseph M Hellerstein, and Ion Stoica. 2014. Coordination avoidance in database systems. *Proceedings of the VLDB Endowment* 8, 3 (2014), 185–196.
- [3] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2006. Boogie: a modular reusable verifier for object-oriented programs. In *Formal Methods for Components and Objects*, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, Gerhard Weikum, Frank S. De Boer, Marcello M. Bonsangue, Susanne Graf, and Willem-Paul De Roeve (Eds.). Vol. 4111. Springer Berlin Heidelberg, Berlin, Heidelberg, 364–387. https://doi.org/10.1007/11804192_17 Series Title: Lecture Notes in Computer Science.
- [4] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2016. The satisfiability modulo theories library (SMT-LIB). Published: tt.smt-lib.org.
- [5] Jan Olaf Blech and Sidi Ould Biha. 2011. Verification of PLC properties based on formal semantics in coq. In *Software Engineering and Formal Methods*, Gilles Barthe, Alberto Pardo, and Gerardo Schneider (Eds.). Springer, Berlin, Heidelberg, 58–73. https://doi.org/10.1007/978-3-642-24690-6_6
- [6] Andrea Cerone, Alexey Gotsman, and Hongseok Yang. 2015. Transaction chopping for parallel snapshot isolation. In *Distributed Computing*, Yoram Moses (Ed.). Springer, Berlin, Heidelberg, 388–404. https://doi.org/10.1007/978-3-662-48653-5_26
- [7] Chameleon. 2025. *Chameleon Cloud*. <https://www.chameleoncloud.org/>
- [8] Austin T. Clements, M. Frans Kaashoek, Nikolai Zeldovich, Robert T. Morris, and Eddie Kohler. 2015. The scalable commutativity rule: designing scalable software for multicore processors. *ACM Transactions on Computer Systems* 32, 4 (2015), 10:1–10:47. <https://doi.org/10.1145/2699681>
- [9] Transaction Processing Performance Council. 2010. *TPC-C Benchmark (Revision 5.11)*. <https://www.tpc.org/tpcc/>
- [10] James Cowling and Barbara Liskov. 2012. Granola: {Low-Overhead} distributed transaction coordination. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. 223–235.
- [11] Joel D. Day, Vijay Ganesh, Paul He, Florin Manea, and Dirk Nowotka. 2018. The satisfiability of word equations: decidable and undecidable theories. In *Reachability Problems*, Igor Potapov and Pierre-Alain Reynier (Eds.). Springer International Publishing, Cham, 15–29. https://doi.org/10.1007/978-3-030-00250-3_2
- [12] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [13] Jose M. Faleiro and Daniel J. Abadi. 2015. Rethinking serializable multiversion concurrency control. *Proceedings of the VLDB Endowment* 8, 11 (July 2015), 1190–1201. <https://doi.org/10.14778/2809974.2809981>
- [14] Jose M Faleiro, Daniel J Abadi, and Joseph M Hellerstein. 2017. High performance transactions via early write visibility. *Proceedings of the VLDB Endowment* 10, 5 (2017).
- [15] Cormac Flanagan and Stephen N. Freund. 2024. Mover Logic: A Concurrent Program Logic for Reduction and Rely-Guarantee Reasoning. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (Leibniz International Proceedings in Informatics (LIPIcs))*, Jonathan Aldrich and Guido Salvaneschi (Eds.), Vol. 313. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 16:1–16:29. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.16>
- [16] Hector Garcia-Molina. 1983. Using semantic knowledge for transaction processing in a distributed database. *ACM Transactions on Database Systems (TODS)* 8, 2 (1983), 186–213.
- [17] Farzad Habibi, Juncheng Fang, Tania Llorido-Botran, and Faisal Nawab. 2025. Brook-2PL: Tolerating High Contention Workloads with A Deadlock-Free Two-Phase Locking Protocol. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–27.
- [18] Ben Johnson. 2019. *An embedded key/value database for Go*. <https://github.com/bolt/bolt>
- [19] Robbert Jan Krebbers. 2015. *The C standard formalized in Coq*. Ph.D. Dissertation. Radboud University Nijmegen. <https://repository.ubn.ru.nl/bitstream/handle/2066/147182/147182.pdf> Publisher: SI: sn.
- [20] Akhil Kumar and Michael Stonebraker. 1988. Semantics based transaction management techniques for replicated data. *ACM SIGMOD Record* 17, 3 (1988), 117–125.
- [21] K. Rustan M. Leino. 2010. Dafny: an automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer, Berlin, Heidelberg, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20
- [22] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno Preguiça, and Rodrigo Rodrigues. 2012. Making Geo-Replicated systems fast as possible, consistent when necessary. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. 265–278.
- [23] Richard J. Lipton. 1975. Reduction: a method of proving properties of parallel programs. *Commun. ACM* 18, 12 (Dec. 1975), 717–721. <https://doi.org/10.1145/361227.361234>
- [24] Shuai Mu, Yang Cui, Yang Zhang, Wyatt Lloyd, and Jinyang Li. 2014. Extracting more concurrency from distributed transactions. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 479–494.
- [25] Dai Qin, Angela Demke Brown, and Ashvin Goel. 2021. Caracal: Contention management with deterministic concurrency control. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 180–194.
- [26] W. V. Quine. 1946. Concatenation as a basis for arithmetic. *Journal of Symbolic Logic* 11, 4 (1946), 105–114. <https://doi.org/10.2307/2268308>
- [27] Zvonimir Rakamarić and Michael Emmi. 2014. SMACK: decoupling source language details from verifier implementations. In *Computer Aided Verification*, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar, Gerhard Weikum, Armin Biere, and Roderick Bloem (Eds.). Vol. 8559. Springer International Publishing, Cham, 106–113. https://doi.org/10.1007/978-3-319-08867-9_7 Series Title: Lecture Notes in Computer Science.
- [28] H. G. Rice. 1953. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. Amer. Math. Soc.* 74, 2 (1953), 358–366. <https://doi.org/10.2307/1990888>
- [29] Sudip Roy, Lucja Kot, Gabriel Bender, Bailu Ding, Hossein Hojjat, Christoph Koch, Nate Foster, and Johannes Gehrke. 2015. The homeostasis protocol: Avoiding transaction coordination through program analysis. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1311–1326.
- [30] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems*, Xavier Défago, Franck Petit, and Vincent Villain (Eds.). Vol. 6976. Springer Berlin Heidelberg, Berlin, Heidelberg, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29 Series Title: Lecture Notes in Computer Science.
- [31] Dennis Shasha, Francois Llirbat, Eric Simon, and Patrick Valduriez. 1995. Transaction chopping: algorithms and performance studies. *ACM Transactions on Database Systems* 20, 3 (Sept. 1995), 325–363. <https://doi.org/10.1145/211414.211427>
- [32] The Rocq Development Team. 2026. *The Rocq proof assistant reference manual*. <https://rocq-prover.org/refman>
- [33] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*. 1–12.
- [34] Alan Mathison Turing. 1936. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* 42, 1 (1936), 230–265.
- [35] Zhaoguo Wang, Shuai Mu, Yang Cui, Han Yi, Haibo Chen, and Jinyang Li. 2016. Scaling Multicore Databases via Constrained Parallel Execution. In *Proceedings of the 2016 International Conference on Management of Data (San Francisco, California, USA) (SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1643–1658. <https://doi.org/10.1145/2882903.2882934>
- [36] Zhaoguo Wang, Shuai Mu, Yang Cui, Han Yi, Haibo Chen, and Jinyang Li. 2016. Scaling multicore databases via constrained parallel execution. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1643–1658. <https://doi.org/10.1145/2882903.2882934>
- [37] William Weihl. 1984. *Specification and Implementation of Atomic Data Types*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [38] Chao Xie, Chunzhi Su, Manos Kapritsos, Yang Wang, Navid Yaghmazadeh, Lorenzo Alvisi, and Prince Mahajan. 2014. Salt: Combining ACID and BASE in a distributed database. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 495–509.
- [39] Yang Zhang, Russell Power, Siyuan Zhou, Yair Sovran, Marcos K. Aguilera, and Jinyang Li. 2013. Transaction chains: achieving serializability with low latency in geo-distributed storage systems. In *Proceedings of the Twenty-fourth ACM Symposium on Operating Systems Principles (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 276–291. <https://doi.org/10.1145/2517349.2522729>